

Introduction

Every developer eventually runs into some version of this problem in real life: a database has gaps in its auto-increment IDs, an invoice sequence skips a few numbers because of a rollback, or a stream of network packets arrives with a chunk missing from the middle. The underlying question is always the same — given a range that *should* be complete, what's missing?

That's exactly why "find missing elements" style questions show up so often in coding interviews. They look simple on the surface, but they're really testing whether you understand the tradeoff between time and space — whether you'll reach for the first working solution or think about what the data structure itself can do for you. If you can explain why a set-based solution costs memory, and then show a way to avoid that cost using the array itself, you've demonstrated exactly the kind of thinking interviewers are trying to screen for.

This problem is also a great entry point into a broader pattern you'll see again and again: using an array's own indices as a lookup table instead of building a separate hash structure. Once you've internalized it here, you'll start recognizing it in completely different problems.

Problem Overview

You're given an array of unique integers. At some point in the past, this array held every integer from its smallest value to its largest value, with no gaps — a complete, unbroken range. Since then, some numbers from the *middle* of that range have been removed. The smallest and largest values are guaranteed to still be present.

Your task: figure out which numbers are missing, and return them as a list sorted from smallest to largest. If nothing is missing, return an empty list.

The constraints worth noting: - All numbers in the input are unique — no duplicates to worry about. - The smallest and largest values in the original range are always present in the array. - Only numbers strictly between the smallest and largest can be missing.

Example

Let's start small. Say `nums = [1, 4, 2, 5]` .

- The smallest value is `1` , the largest is `5` .

- The complete range should therefore be 1, 2, 3, 4, 5.
- Checking each number against `nums`: 1 ✓, 2 ✓, 3 X, 4 ✓, 5 ✓.

Only 3 is missing, so the expected output is `[3]`.

A second example makes the "nothing missing" case concrete: `nums = [6, 7, 8, 9]`. The range is 6 through 9, and every one of those values is already present. The expected output is `[]`.

Intuition

Before writing any code, it helps to think like someone who's solved a dozen variations of this problem already.

The most natural instinct is: for every number in the full range, ask "is this in my array?" That's a perfectly valid mental model, and it's where almost everyone starts. The question is *how* you answer "is this in my array?" efficiently.

Checking membership against a plain Python list means scanning the whole list every time — that's $O(n)$ per check, and $O(n)$ checks total, so $O(n^2)$ overall. The obvious fix is a `set`, which gives you $O(1)$ average-case membership checks and drops the whole thing to $O(n)$ time. That's a completely reasonable solution, and for most day-to-day scripts, it's the one you should write.

But an experienced engineer will ask a follow-up question: *do I actually need a second data structure at all?* A set costs extra memory proportional to the input size. If the interview (or the constraints) specifically call for $O(1)$ extra space, you need a different idea.

Here's the insight: instead of asking a hash structure "have I seen this number," you can let the *position* in an array answer that question for you. If you build an array called `tracker` where slot 0 represents the smallest number in the range, slot 1 represents the next number up, and so on, then the index itself becomes a lookup — no searching required. Given any number `num` and the range's starting value `lo`, the slot it belongs to is simply `num - lo`. That's the entire trick: **the index acts as the hash, and the sign of the value at that index acts as the "have I seen this" flag.**

Brute Force Solution

Idea: For every number in the full range from `lo` to `hi`, check whether it exists in `nums`. If it doesn't, it's missing.

Algorithm: 1. Compute `lo = min(nums)` and `hi = max(nums)`. 2. Loop through every integer from `lo` to `hi`. 3. For each one, check membership in `nums` using `in`. 4. Collect the ones that

aren't found.

Advantages: Extremely easy to read and reason about. No tricky index math.

Disadvantages: Checking `x in nums` against a plain list scans the entire list each time, making this $O(\text{range} \times n)$ in the worst case — slow for large inputs. Upgrading to `x in set(nums)` fixes the speed problem but costs $O(n)$ extra memory for the set.

Time complexity: $O(n)$ with a set, $O(n \times \text{range})$ with a plain list. **Space complexity:** $O(n)$ for the set.

Optimal Solution

The optimal approach reuses one array — no separate hash set — by treating each slot's sign as a marker.

Step 1 — Build the tracker. Compute `lo` and `hi`, then build `tracker = list(range(lo, hi + 1))`. Slot `i` holds the value `lo + i`, so the value stored is the index's meaning until we mark it.

Step 2 — First pass, mark what you've seen. Walk through `nums`. For each `num`, compute `idx = num - lo` — that's the slot this number owns. Flip `tracker[idx]` to negative. Negative now means "this value showed up in `nums`."

Step 3 — Second pass, collect what's untouched. Walk through `tracker`. Any slot still positive was never visited during marking — meaning that value never appeared in `nums`, so it's missing.

Because `tracker` was built in increasing order from `lo` to `hi`, the surviving positive values come out already sorted — no extra sort step needed.

A visual for `nums = [1, 4, 2, 5]` (`lo = 1`, `hi = 5`):

Index	0	1	2	3	4
Initial value	1	2	3	4	5
After marking	-1	-2	3	-4	-5

Index `2` never gets flipped — that's your missing number, `3`, hiding in plain sight.

Python Code

```
def find_missing_elements(nums: list[int]) -> list[int]:
    """Return the numbers missing from the complete range [min(nums), max(nums)]."""
    lo, hi = min(nums), max(nums)
```

```
tracker = list(range(lo, hi + 1))

# Pass 1: mark every number that appears by flipping its slot negative.
for num in nums:
    idx = num - lo
    tracker[idx] = -abs(tracker[idx])

# Pass 2: whatever is still positive was never marked, so it's missing.
return [val for val in tracker if val > 0]
```

Code Walkthrough

- `lo, hi = min(nums), max(nums)` — finds the boundaries of the range in a single pass each.
- `tracker = list(range(lo, hi + 1))` — builds the full expected range as an actual list. The `+1` matters because `range` stops one short of its second argument.
- `for num in nums:` — the marking pass. Each number in the input tells us exactly which slot to touch.
- `idx = num - lo` — converts a value into a position. This is "how far is this number from the start of the range," which is exactly which slot represents it.
- `tracker[idx] = -abs(tracker[idx])` — flips that slot negative. Wrapping in `abs()` first guarantees the result is always freshly negative, so even if a slot got touched more than once it stays negative rather than flipping back positive.
- `[val for val in tracker if val > 0]` — a list comprehension that keeps only the untouched (still-positive) slots. Since `tracker` was built in ascending order, this list comes out sorted for free.

Dry Run

Let's trace `nums = [5, 1]` by hand.

1. `lo = min([5, 1]) = 1`, `hi = max([5, 1]) = 5`.
2. `tracker = list(range(1, 6))` → `[1, 2, 3, 4, 5]`.
3. First iteration, `num = 5`: `idx = 5 - 1 = 4`. Flip slot 4 → `tracker = [1, 2, 3, 4, -5]`.
4. Second iteration, `num = 1`: `idx = 1 - 1 = 0`. Flip slot 0 → `tracker = [-1, 2, 3, 4, -5]`.
5. Loop ends. Collect positive values: `2, 3, 4`.

Output: `[2, 3, 4]` — every number between `1` and `5` that never appeared in the input.

Complexity Analysis

- **Time — $O(n)$:** The marking pass touches every number in `nums` exactly once. The collection pass touches every slot in `tracker` exactly once. Both are single straight-through loops with no nesting, so the work scales linearly with the size of the range.
- **Space — $O(1)$ extra:** `tracker` itself is proportional to the range size, but it doubles as the output — we never allocate a second structure like a hash set purely for lookups. If you don't count the output array (a common convention in these problems), the auxiliary space is constant.

This is strictly better on memory than the set-based approach while matching it on time.

Alternative Solutions

The simplest alternative uses a set directly:

```
def find_missing_elements_set(nums: list[int]) -> list[int]:  
    lo, hi = min(nums), max(nums)  
    seen = set(nums)  
    return [n for n in range(lo, hi + 1) if n not in seen]  
end
```

Wait — that's three lines and reads instantly: build a set, sweep the range, keep what's absent. For a quick script or a one-off tool, this is genuinely the better choice — it's obviously correct at a glance, and readability usually beats a marginal memory saving. The in-place tracker trick earns its complexity specifically when you need to avoid the extra $O(n)$ memory a set requires, which matters more in constrained environments (embedded systems, very large inputs, strict interview constraints) than in everyday code.

Edge Cases

- **Nothing missing:** `nums = [6, 7, 8, 9]` — every slot gets flipped, the comprehension finds nothing positive, returns `[]`.
- **Only two numbers (just `lo` and `hi`):** `tracker` has two slots, both get marked, result is `[]`.
- **Range of size one (`lo == hi`):** A single-element input like `[5]` gives `tracker = [5]`, which gets flipped to `[-5]`, correctly returning `[]`.
- **Uniqueness guarantee:** Since the problem promises no duplicates in `nums`, no slot is ever flipped twice by two different values — the `abs()` guard is a safety net, not something the given

constraints require you to lean on.

- **Large gaps:** If most of the middle is missing, `tracker` will have many positive survivors — the comprehension handles any proportion correctly since it's just filtering.

Common Mistakes

1. **Forgetting to subtract `lo` before indexing.** Using the raw value as an index only works if the range starts at `0` or `1`. The moment `lo` is something like `50`, indexing with the raw value goes out of bounds or points at the wrong slot.
2. **Off-by-one on the tracker size.** The tracker needs `hi - lo + 1` slots, not `hi - lo`. Forgetting the `+1` silently drops the last value in the range.
3. **Flipping without `abs()` first.** If you ever process a value more than once (duplicates, or a modified version of this problem), negating an already-negative slot flips it back positive and corrupts your marking.
4. **Sorting the output unnecessarily.** Since `tracker` is already built in ascending order, running `sorted()` on the result is wasted work — a sign you haven't fully internalized why the trick produces sorted output for free.
5. **Assuming the array is already sorted.** The input order of `nums` doesn't matter for this approach — trying to rely on input order (e.g., checking adjacent differences) breaks the moment the array isn't presented in sorted form.

Interview Questions

- What if the array could contain duplicates — how would you adapt the marking step?
- How would you solve this if you couldn't modify the input array in place?
- Can you find the missing elements without ever materializing the full range?
- How does this "index as hash" pattern relate to cyclic sort or Floyd's cycle detection?
- What happens to your solution's complexity if the range is enormous but `nums` is sparse within it?

Similar Problems

- **Find the Duplicate Number (LeetCode 287)** — same "index as hash" family, but you're hunting for a value that appears twice instead of one that's absent.
- **First Missing Positive (LeetCode 41)** — a harder variant of the same in-place marking idea, applied to unsorted arrays with negative numbers and no guaranteed range.

- **Missing Number (LeetCode 268)** — a simpler cousin where the range is always `0` to `n`, solvable with sum arithmetic or XOR as well as this indexing trick.
- **Set Mismatch (LeetCode 645)** — combines "find the missing number" with "find the duplicate number" in a single pass.

Key Takeaways

The core lesson here isn't really about missing numbers — it's about recognizing when an array's own structure can replace a second data structure. The index becomes your lookup, the sign becomes your "have I seen this" flag, and the array you already have doubles as your answer. That's a pattern worth carrying into every future problem where you catch yourself reaching for a set out of habit rather than necessity.

Watch the Video

If you want to see this built up visually — the marking animation, the pause-and-solve challenge, and the full dry run — watch the video here: {LINK}

About the Series

This is part of Fun with Learning Technology, a daily series working through LeetCode problems the way you'd actually want them explained in an interview: starting from the obvious solution, understanding exactly why it's not good enough, and building up to the version that shows real engineering judgment.

Call To Action

If this walkthrough helped it click, the like and subscribe buttons on YouTube go a long way toward keeping this series going. For deeper dives and the problems that don't make it to video, subscribe to the Substack. And if there's a topic you want explained next, drop it in the comments — that's genuinely how a lot of these episodes get picked.

Quick Review

What tells you to use the index-as-hash-key trick?

Array holds numbers in a bounded range (1..n) and you need to find missing/duplicate values without extra space.

Time complexity of the index-marking approach?

$O(n)$ — two passes over the array, no nested loops.

Space complexity of the index-marking approach?

$O(1)$ extra space — the input array itself stores the 'seen' flags.

What's the trickiest bug when marking indices negative?

Forgetting to take $\text{abs}(\text{value})$ before using it as an index, since earlier marks already flipped the sign.

Brute-force check-each-number vs index-as-hash approach?

Brute-force uses a set for $O(n)$ space; index-marking reuses the array for $O(1)$ space, same $O(n)$ time.

Follow-up: array isn't 1..n range, still find missing values?

Fall back to a hash set/count map — the index trick only works when values map cleanly to valid indices.